Corso di Laurea in Informatica
Anno Accademico 25/26

Gioco del Tris distribuito: progettazione e sviluppo con client-server containerizzato

Autori del Progetto

Giuseppe Pio Caruso
N86004231
giuseppep.caruso@studenti.unina.it

Danilo Del Prete
N86004956
danil.delprete@studenti.unina.it

Indice

1 Introduzione **3**
1.1 Struttura del progetto . . . 3

2 Architettura e Funzionalità **4**
2.1 Descrizione delle Funzionalità . . . 4
2.1.1 Connessione e Identificazione Utente . . . 4
2.1.2 Sala D'Attesa . . . 4
2.1.3 Ciclo di gioco . . . 4

3 Responsabilità Client-Server **5**
3.1 Responsabilità del Server . . . 5
3.2 Responsabilità del Client . . . 5

4 Gestione dei Thread e Sincronizzazione **7**
4.1 Architettura Multi-Thread . . . 7
4.2 Protezione delle Strutture Dati Condivise . . . 7
4.3 Sincronizzazione tramite Condition Variables . . . 7
4.4 Signal Handlers e Notifiche Asincrone . . . 7
4.5 Race Conditions e Gestione della Memoria . . . 8

5 Deployment con Docker **9**
5.1 Containerizzazione del Sistema . . . 9
5.2 Dockerfile del Server . . . 9
5.3 Dockerfile del Client . . . 9
5.4 Docker Compose . . . 10
5.5 Scaling dei Client . . . 10

6 Automazione e Utilizzo del Sistema **11**
6.1 Script di Avvio Automatizzato . . . 11
6.2 Sincronizzazione e Apertura dei Terminali . . . 11
6.3 Vantaggi e Limitazioni . . . 11

Capitolo 1

Introduzione

Questo elaborato descrive la realizzazione di un sistema Client-Server per giocare a tris in modalità concorrente. L'obiettivo è illustrare le scelte di progettazione e implementazione adottate durante lo sviluppo.
Il linguaggio scelto è C per implementare sia il Client che il Server, mentre l'ambiente di esecuzione è configurato tramite Docker Compose. L'avvio dell'applicazione è automatizzato mediante script che si adattano al sistema operativo in uso.

1.1 Struttura del progetto

```
tris-project/
├── client/
│   ├── connessione.c
│   ├── connessione.h
│   ├── Dockerfile
│   ├── gioco.c
│   ├── gioco.h
│   ├── interfaccia.c
│   ├── interfaccia.h
│   ├── main.c
│   ├── Makefile
│   └── structure.h
├── server/
│   ├── connessione.c
│   ├── connessione.h
│   ├── Dockerfile
│   ├── gestione_client.c
│   ├── gestione_client.h
│   ├── gestione_partita.c
│   ├── gestione_partita.h
│   ├── main.c
│   ├── Makefile
│   └── structure.h
├── docker-compose.yml
├── projectUnix.sh
└── projectWindows.ps1
```

Capitolo 2

Architettura e Funzionalità

2.1 Descrizione delle Funzionalità

Come primo step, il server crea un thread univoco per ogni client, quindi per ogni giocatore.
Il thread ha il compito di gestire le tre fasi del progetto: registrazione, sala d'attesa e partita.

2.1.1 Connessione e Identificazione Utente

Da qui parte il gioco: ogni client è tenuto a registrarsi con un nome proprio con cui verranno identificati, quindi i nomi non possono essere uguali. Ovviamente vi saranno dei controlli sul nome inserito dal giocatore.
La registrazione è l'unica delle tre fasi a verificarsi soltanto una volta.

2.1.2 Sala D'Attesa

Una volta registrati, gli utenti verranno inseriti in una stanza comune.
All'interno di questa stanza si può visualizzare il proprio storico, si possono creare partite o ci si può unire a partite già esistenti.
Quando un utente effettua l'accesso in lobby, gli altri utenti ricevono a schermo una notifica del tipo "NomeUtente è entrato in lobby".
Si può decidere di creare una partita scrivendo "crea" o di unirsi, qualora esistesse una partita, scrivendo l'ID relativo ad essa. Ovviamente quando viene creata una partita, gli altri utenti possono vederne i dettagli, come nome del proprietario, stato, id.
Questa fase di lobby può ripetersi più volte durante una sessione dato che, terminata una partita, si può far ritorno a questa stanza.

2.1.3 Ciclo di gioco

Come scritto pocanzi, un utente può decidere di creare una partita o di unirsi ad una già creata. Analizziamo entrambi gli scenari:
L'utente crea la partita scrivendo "crea" e resta in attesa di un avversario. Quando quest'ultimo intende unirsi, il proprietario riceve a schermo un messaggio di accettazione del tipo "NomeUtente vuole unirsi, accetti? (s/n)", con "s" si accetta e con "n" si rifiuta.
In caso di esito positivo, la partita comincia: a turno inseriscono un numero da 0 a 9 relativo alla casella che intendono occupare con la propria mossa. Per capire ogni numero a che casella corrisponde, a schermo sarà sempre presente un ulteriore campo di gioco di esempio.
La partita può avere tre esiti: vittoria, pareggio o sconfitta. A partita terminata, Il vincitore può decidere se cercare un nuovo avversario (diventando proprietario della partita se non lo è già) oppure tornare alla lobby, mentre il perdente torna in ogni caso alla lobby. In caso di pareggio, gli utenti possono lanciarsi una rivincita invertendo l'ordine del primo turno; se almeno uno dei due dovesse rifiutare, il proprietario può cercare una nuova partita e l'avversario torna alla lobby.

Capitolo 3

Responsabilità Client-Server

3.1 Responsabilità del Server

Come illustrato precedentemente, alla connessione di un nuovo giocatore il server crea un thread dedicato per gestirlo. Il processo server gestisce i dati relativi ai giocatori e alle partite tramite le liste globali `testa_giocatori` e `testa_partite`, accessibili da ogni thread mediante mutex per garantire operazioni thread-safe.
Ciascun thread gestisce l'intero ciclo di interazione con il giocatore-client, dalla registrazione fino alla disconnessione. In fase di registrazione, il thread riceve il nome del giocatore, verifica l'univocità confrontandolo con i giocatori già registrati, e alloca memoria per il nuovo `nodo_giocatore`. Durante questa fase vengono inizializzati lo stato del giocatore e le primitive di sincronizzazione.
In fase di lobby, il thread invia le statistiche personali del giocatore, comunica la lista delle partite disponibili con i relativi ID univoci e permanenti, notifica l'ingresso di nuovi giocatori, e gestisce gli aggiornamenti tramite il segnale `SIGUSR1`. Processa i comandi del giocatore (creazione partita, richiesta di unione, disconnessione) e segnala al client il momento di ingresso in partita comunicando il ruolo assegnato.
Durante la partita, la comunicazione tra i due giocatori viene gestita dal thread del proprietario che funge da intermediario, mentre il thread dell'avversario rimane in attesa passiva su una condition variable. Per mantenere il server leggero, esso non gestisce la logica di gioco ma si limita a ritrasmettere le mosse da un client all'altro senza validazioni, ricevere l'esito comunicato dai client, e gestire errori di rete o disconnessioni.
Al termine della partita, il thread del proprietario aggiorna le statistiche dei giocatori e decide la successiva azione in base all'esito: comunicare il ritorno in lobby notificando gli altri giocatori, gestire richieste di rivincita in caso di pareggio, o assegnare il titolo di "campione" al vincitore. Il server garantisce la persistenza degli ID delle partite tramite un contatore globale incrementale, assicurando che ogni partita riceva un ID univoco non riutilizzato.

3.2 Responsabilità del Client

Il client è responsabile dell'intera logica di gioco e della gestione dell'interfaccia utente. Questa scelta progettuale permette al server di rimanere leggero e scalabile, delegando al client tutte le operazioni relative al tris.
Il client implementa completamente le regole del gioco, validando le mosse inserite dall'utente (casella valida tra 1 e 9 e non occupata), aggiornando la griglia locale, e calcolando autonomamente l'esito controllando le condizioni di vittoria o pareggio. L'esito viene poi comunicato al server che lo ritrasmette all'avversario senza verifiche.
Gestisce completamente la visualizzazione mostrando la griglia aggiornata dopo ogni mossa, distinguendo le caselle del giocatore ('O') da quelle dell'avversario ('X'), e visualizzando la numerazione delle caselle disponibili. Durante la lobby, mostra le statistiche personali e la lista delle partite con ID univoci, oltre ai messaggi di sistema come l'ingresso di nuovi giocatori o terminazione di partite altrui.
Il client utilizza un thread dedicato per la lettura asincrona dell'input utente tramite `fgets()`, validando formato e lunghezza prima dell'invio al server. Il thread principale rimane in ascolto sul socket per i messaggi del server, garantendo reattività. Durante la partita, il thread di input viene temporaneamente terminato tramite

`SIGUSR1` ed ricreato al ritorno in lobby.
Comunica costantemente con il server inviando mosse ed esiti, ricevendo le mosse avversarie, e gestendo i flag di controllo (`NO_ERROR` o `ERROR_CONN`) per rilevare disconnessioni. Gestisce autonomamente richieste di rivincita e scelte post-partita, interpretando i messaggi del server per distinguere tra notifiche, aggiornamenti liste, e inizio partita con relativo ruolo assegnato.

Capitolo 4

Gestione dei Thread e Sincronizzazione

4.1 Architettura Multi-Thread

Il server adotta un'architettura multi-thread dove ogni connessione client viene gestita da un thread dedicato. Tutti i thread vengono creati in modalità detached tramite `pthread_create()`, poiché il processo principale non necessita di conoscerne lo stato di terminazione. Questo approccio semplifica la gestione della memoria e rende il sistema più scalabile, evitando la necessità di chiamate `pthread_join()`.
All'avvio, il processo server inizializza le strutture dati globali e configura i signal handler. Nel ciclo principale, accetta nuove connessioni tramite `accept()`, alloca dinamicamente memoria per il socket descriptor, e lo passa al thread appena creato. Quest'ultimo gestisce l'intera interazione con il giocatore-client, dalla registrazione fino alla disconnessione.

4.2 Protezione delle Strutture Dati Condivise

Il server utilizza due liste concatenate protette da mutex: `testa_giocatori` per i giocatori connessi e `testa_partite` per le partite attive. Ogni nodo giocatore viene allocato dal thread associato in fase di registrazione, mentre un nodo partita viene allocato alla creazione della partita e cancellato quando termina definitivamente.
I mutex `mutex_giocatori` e `mutex_partite`, allocati staticamente, atomizzano le operazioni critiche sulle rispettive liste (inserimento, ricerca, rimozione e aggiornamento). È fondamentale rilasciare sempre il lock prima di operazioni bloccanti come `send()` o `recv()` per evitare deadlock.

4.3 Sincronizzazione tramite Condition Variables

Le condition variables sincronizzano thread che devono attendere eventi specifici. Quando un giocatore crea una partita, il thread del proprietario si blocca sulla variabile `stato_cv` della struttura `nodo_partita` tramite `pthread_cond_timedwait()`, che permette verifiche periodiche della connessione tramite timeout. L'uso del timeout evita che il thread rimanga bloccato indefinitamente in caso di disconnessione del proprietario.
Quando un avversario si unisce e viene accettato, lo stato della partita diventa `IN_CORSO` e `pthread_cond_signal()` risveglia il thread del proprietario. Durante la partita, il thread dell'avversario attende sulla propria condition variable `stato_cv` associata a `nodo_giocatore`. Al termine, il proprietario aggiorna lo stato dell'avversario e lo sblocca con `pthread_cond_signal()`.
Oltre ai mutex associati alle condition variables di ciascun giocatore e partita, i due mutex globali delle liste garantiscono accessi thread-safe alle strutture dati condivise.

4.4 Signal Handlers e Notifiche Asincrone

Il sistema sfrutta segnali POSIX per notificare eventi asincroni. I signal handler sono configurati con il flag `SA_RESTART` per evitare interruzioni indesiderate delle chiamate di sistema bloccanti.

La funzionalità di aggiornamento dei giocatori in lobby quando le partite cambiano stato è implementata tramite `SIGUSR1`. La funzione `notifica_aggiornamento_partite()` invia il segnale con `pthread_kill()` a tutti i thread in stato `IN_LOBBY` escluso il mittente. L'handler `invia_lista_partite_disponibili()` invia automaticamente la lista aggiornata al client, mantenendo la consistenza delle informazioni.
Il segnale `SIGALRM` gestisce timeout e disconnessioni. Invocato da `gestisci_errore_connessione()` tramite `pthread_kill()`, il relativo handler esegue il cleanup (chiusura socket, rimozione dalla lista) e termina il thread con `pthread_exit()`.
`SIGTERM`, inviato da Docker durante lo shutdown del container, permette al server di terminare in modo pulito chiudendo le socket e velocizzando la chiusura che altrimenti impiegherebbe diversi secondi.

4.5 Race Conditions e Gestione della Memoria

Durante lo sviluppo sono state risolte diverse race conditions critiche. Il passaggio del socket descriptor ai thread, inizialmente per riferimento, causava problemi quando il ciclo di `accept()` sovrascriveva la variabile. Risolto allocando dinamicamente memoria per ogni socket. Il controllo di unicità del nome richiede atomicità tra verifica e inserimento, mantenendo il lock durante entrambe le operazioni. L'iterazione sulla lista partite durante modifiche concorrenti poteva causare accessi a memoria deallocata, risolto acquisendo il lock per l'intera iterazione.
Ogni thread dealloca le risorse allocate: il socket descriptor viene liberato dopo la lettura, il nodo giocatore alla disconnessione, e le partite al termine dopo aver distrutto mutex e condition variables tramite `pthread_mutex_destroy()` e `pthread_cond_destroy()`.

Capitolo 5

Deployment con Docker

5.1 Containerizzazione del Sistema

Il progetto utilizza Docker per la containerizzazione sia del server che dei client, garantendo portabilità, isolamento e facilità di deployment. La configurazione si basa su tre componenti principali: il Dockerfile del server, il Dockerfile del client, e il file `docker-compose.yml` che orchestra l'intero sistema.

5.2 Dockerfile del Server

Il Dockerfile del server utilizza Ubuntu come immagine base e configura l'ambiente necessario per la compilazione ed esecuzione:

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y gcc make
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY . .
RUN make
EXPOSE 8080
CMD ["./server"]
```

Il processo di build installa gli strumenti di compilazione (`gcc` e `make`), rimuovendo successivamente le liste dei pacchetti per ridurre la dimensione dell'immagine. La directory di lavoro viene impostata a `/app` dove vengono copiati tutti i file sorgente. Il comando `RUN make` compila il codice durante la fase di build dell'immagine, non ad ogni avvio del container, ottimizzando i tempi di startup.
L'istruzione `EXPOSE 8080` documenta che il container espone la porta 8080 per la comunicazione TCP, sulla quale il server si mette in ascolto. Infine, `CMD` specifica l'eseguibile da lanciare all'avvio del container.

5.3 Dockerfile del Client

Il Dockerfile del client segue una struttura analoga al server, con alcune differenze significative:

```
FROM ubuntu:latest
RUN apt-get update && apt-get install -y gcc make
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY . .
RUN make
CMD ["./client"]
```

La principale differenza rispetto al server è l'assenza dell'istruzione `EXPOSE`, poiché il client non riceve connessioni in ingresso ma si connette attivamente al server. Il sistema operativo assegna automaticamente

una porta effimera disponibile per la connessione uscente, senza necessità di configurazione esplicita tramite `bind()`.

5.4 Docker Compose

Il file `docker-compose.yml` coordina l'avvio e la configurazione di server e client, definendo le dipendenze e il networking tra i container:

```
services:
  server:
    build:
      context: ./Server
    container_name: server

  client:
    build:
      context: ./Client
    stdin_open: true
    tty: true
    depends_on:
      - server
    network_mode: "container:server"
```

Il servizio `server` specifica il contesto di build nella directory `./Server` e assegna esplicitamente il nome `server` al container, facilitandone il riferimento da parte dei client.
Il servizio `client` presenta configurazioni aggiuntive. Le opzioni `stdin_open: true` e `tty: true` mantengono rispettivamente lo standard input aperto e abilitano un terminale interattivo, essenziali per permettere l'input dell'utente durante il gioco. La direttiva `depends_on` garantisce che Docker avvii prima il server, sebbene non attenda che il server sia effettivamente pronto ad accettare connessioni (solo che il container sia started).
La configurazione `network_mode: "container:server"` è cruciale: fa sì che il client condivida lo stesso network namespace del server. Questo permette al client di connettersi al server utilizzando `localhost` (127.0.0.1) come se fossero sulla stessa macchina, semplificando la configurazione di rete e eliminando la necessità di scoprire l'indirizzo IP del container server.

5.5 Scaling dei Client

Docker Compose permette di scalare facilmente il numero di client tramite il comando:

```
docker-compose up --scale client=N
```

dove `N` rappresenta il numero di istanze client desiderate. Ogni istanza viene eseguita in un container separato con nome univoco (`client-1`, `client-2`, ecc.), ma tutte condividono il network namespace del server.
Per facilitare l'interazione con più client, è stato sviluppato uno script PowerShell che automatizza l'avvio del sistema e l'apertura di terminali separati per ogni client, permettendo agli utenti di interagire simultaneamente con le diverse istanze senza dover eseguire manualmente comandi `docker attach`.

Capitolo 6

Automazione e Utilizzo del Sistema

6.1 Script di Avvio Automatizzato

L'avvio dell'applicazione è completamente automatizzato tramite uno script PowerShell per Windows (`projectWindows.ps1`), progettato per semplificare l'esperienza utente eliminando la necessità di interagire manualmente con i comandi Docker.
Lo script richiede inizialmente all'utente di specificare il numero di client da avviare, quindi esegue `docker-compose up --build --scale client=N` in una nuova finestra PowerShell. Docker Compose avvia un container server e N container client secondo le configurazioni specificate nei Dockerfile e nel file `docker-compose.yml`.

6.2 Sincronizzazione e Apertura dei Terminali

Mentre Docker Compose costruisce ed avvia i container, lo script entra in un ciclo di attesa che verifica periodicamente (ogni 2 secondi) tramite `docker ps` che tutti i container richiesti siano effettivamente in esecuzione. Questo meccanismo garantisce che i container siano completamente inizializzati prima di procedere, evitando errori dovuti a tentativi di connessione prematura.
Una volta verificata la presenza di tutti i container, lo script apre automaticamente una finestra PowerShell separata per ogni client, eseguendo `docker attach` per collegare ciascuna finestra al rispettivo container. L'opzione `-NoExit` mantiene aperta la finestra anche dopo la terminazione del container, permettendo di visualizzare eventuali messaggi di errore. Ogni finestra viene automaticamente posizionata dal sistema operativo, consentendo agli utenti di interagire simultaneamente con più client.

6.3 Vantaggi e Limitazioni

L'automazione tramite script offre diversi benefici significativi. Elimina completamente la necessità di conoscere i comandi Docker, rendendo l'applicazione accessibile anche a utenti non tecnici. La gestione simultanea di più client diventa immediata e intuitiva, con ogni istanza accessibile in una finestra dedicata. In fase di sviluppo, questa automazione si è rivelata fondamentale per testare rapidamente scenari con molti giocatori connessi.
Lo script è stato progettato specificamente per Windows. Per sistemi Unix-like, sarebbe necessario adattare i comandi di apertura terminale utilizzando emulatori nativi come `gnome-terminal`, `konsole` o `xterm`. Su Windows, gli utenti potrebbero dover modificare temporaneamente la policy di esecuzione degli script PowerShell tramite `Set-ExecutionPolicy RemoteSigned` per permettere l'esecuzione dello script.